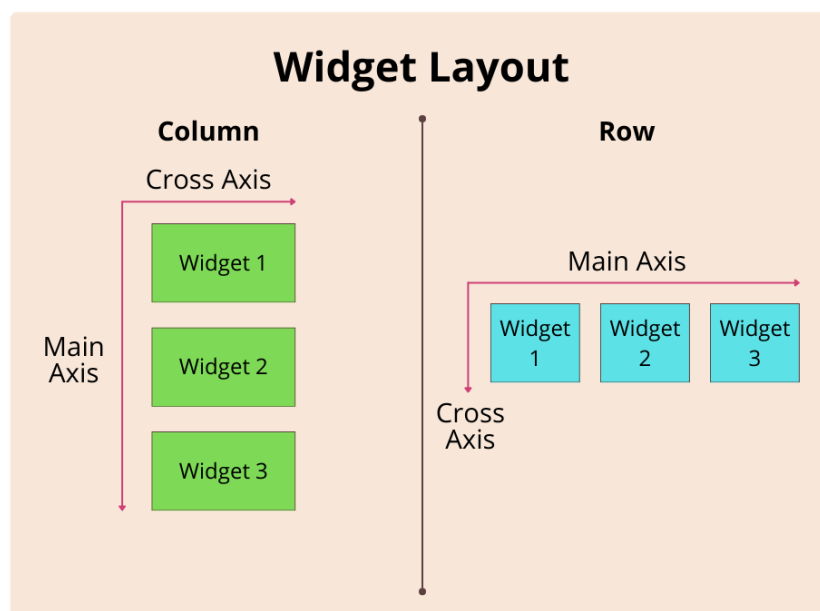


- width & height → Sets container size.
  - margin: EdgeInsets.all(10) → Adds space outside.
  - padding: EdgeInsets.all(15) → Adds space inside.
  - decoration → Applies background, border, shadow, and rounded corners.
  - child → Contains a Text widget inside the container.
- e. Why Use the Container Widget?
- Main Benefits:
    - Adds spacing with margin and padding.
    - Styles elements with borders, colors, and shadows.
    - Wraps content to control size and positioning.

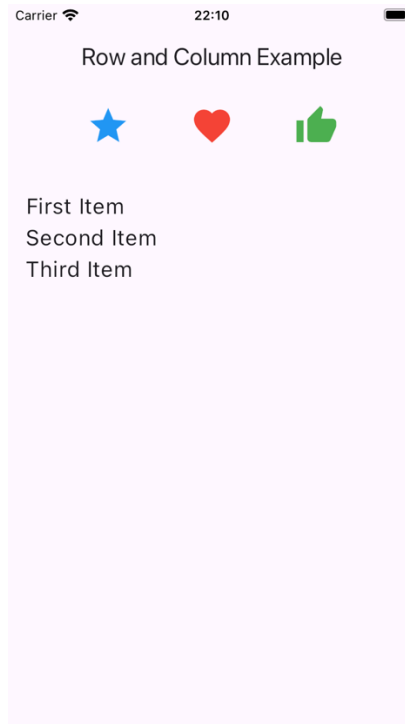
## Row and Column Widget

- a. What are Row & Column?
- Used to arrange widgets in a horizontal or vertical layout.
  - Row → Aligns widgets horizontally (side by side).
  - Column → Aligns widgets vertically (one on top of another).
- b. Understanding Main Axis & Cross Axis
- When using Row or Column, there are two important axes:

| Axis Type  | Row                                                                     | Column                                                                    |
|------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Main Axis  | <b>Horizontal (左⇌右)</b> → Widgets are placed <b>side by side</b> .      | <b>Vertical (上⇕下)</b> → Widgets are placed <b>one below another</b> .     |
| Cross Axis | <b>Vertical (上⇕下)</b> → Controls alignment <b>top, center, bottom</b> . | <b>Horizontal (左⇌右)</b> → Controls alignment <b>left, center, right</b> . |



- c. Example: Row & Column in Action



#### ▪ Row Example (Horizontal Layout)

```
// Row Example
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, //
  Aligns widgets horizontally
  crossAxisAlignment: CrossAxisAlignment.center,    //
  Aligns widgets vertically
  children: [
    Icon(Icons.star, size: 40, color: Colors.blue),
    Icon(Icons.favorite, size: 40, color: Colors.red),
    Icon(Icons.thumb_up, size: 40, color: Colors.green),
  ],
),
```

#### ▪ How It Works

- `mainAxisAlignment.spaceEvenly` → Spaces icons evenly along horizontal axis.
- `crossAxisAlignment.center` → Centers icons vertically.
- Three icons (`Icon(Icons.star)`, etc.) are arranged side by side.

#### ▪ Column Example (Vertical Layout)

```
// Column Example
Column(
  mainAxisAlignment: MainAxisAlignment.center,    // Aligns
  widgets vertically
  crossAxisAlignment: CrossAxisAlignment.start,  // Aligns
  widgets horizontally
  children: [
    Text('First Item', style: TextStyle(fontSize: 20)),
    Text('Second Item', style: TextStyle(fontSize: 20)),
  ],
),
```

```

    Text('Third Item', style: TextStyle(fontSize: 20)),
  ],
),

```

- How It Works
  - `mainAxisAlignment.center` → Centers text widgets vertically.
  - `crossAxisAlignment.start` → Aligns text to the left.
  - Three text widgets (`Text('First Item')`, etc.) appear one below another.
- d. Why are Row & Column Different?
  - Key Differences:

| Feature                 | Row                                                               | Column                                                              |
|-------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Layout Direction</b> | <b>Horizontal (横)</b> → Widgets placed <b>side by side</b> .      | <b>Vertical (縦)</b> → Widgets placed <b>one below another</b> .     |
| <b>Main Axis</b>        | <b>Horizontal (左⇌右)</b> → Controls spacing between widgets.       | <b>Vertical (上⇕下)</b> → Controls spacing between widgets.           |
| <b>Cross Axis</b>       | <b>Vertical (上⇕下)</b> → Controls alignment (top, center, bottom). | <b>Horizontal (左⇌右)</b> → Controls alignment (left, center, right). |

## Button Widget

- a. What are Buttons in Flutter?
  - Interactive widgets that trigger actions when clicked.
  - Essential for navigation, form submission, and user interaction.
  - Different button types for different use cases.
- b. Types of Buttons in Flutter
  - Flutter provides three main types of buttons:

| Button Type                 | Description                             | When to Use                         |
|-----------------------------|-----------------------------------------|-------------------------------------|
| <code>TextButton</code>     | Flat button with no elevation.          | Less prominent actions.             |
| <code>ElevatedButton</code> | Raised button with elevation.           | Highlight important actions.        |
| <code>OutlinedButton</code> | Button with a border but no background. | Secondary or less critical actions. |

- c. Why is `onPressed` Important?

